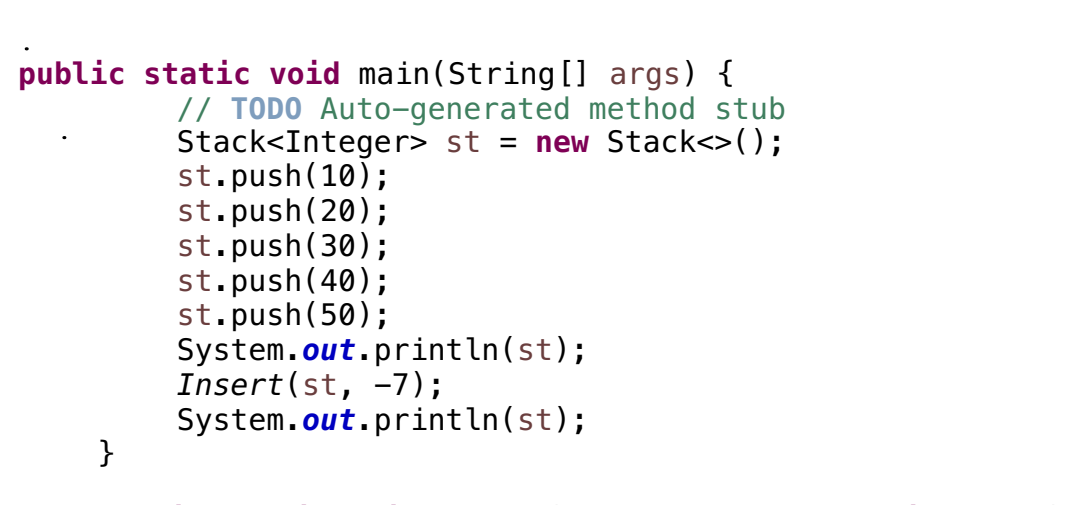
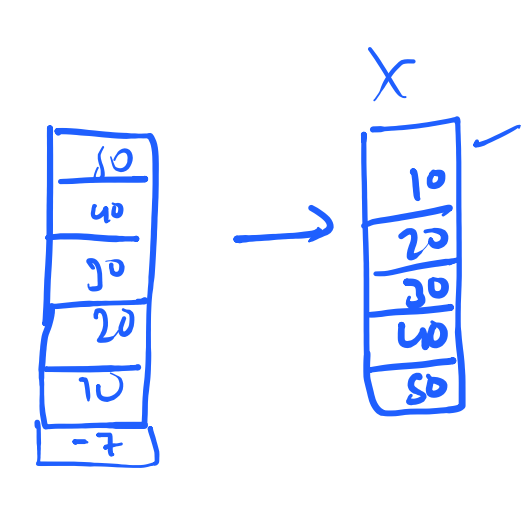
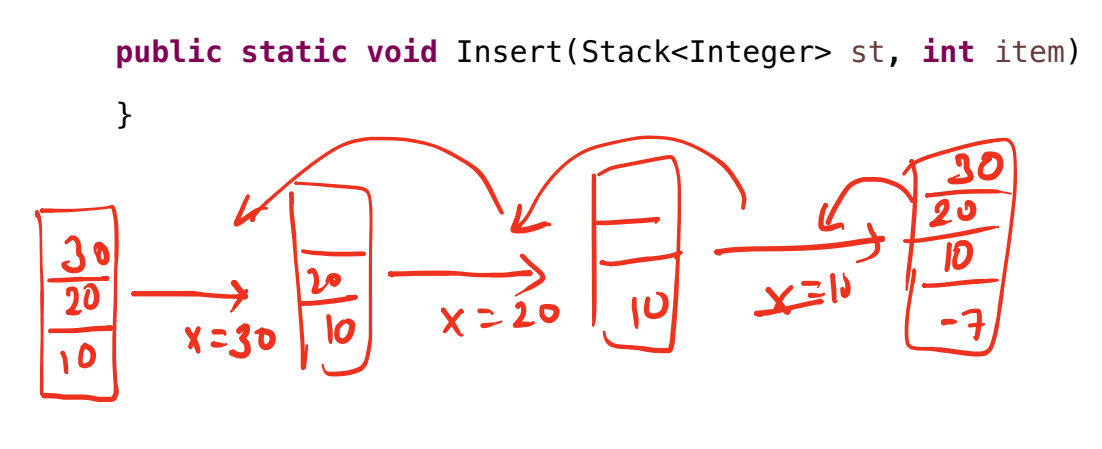
Lec-17

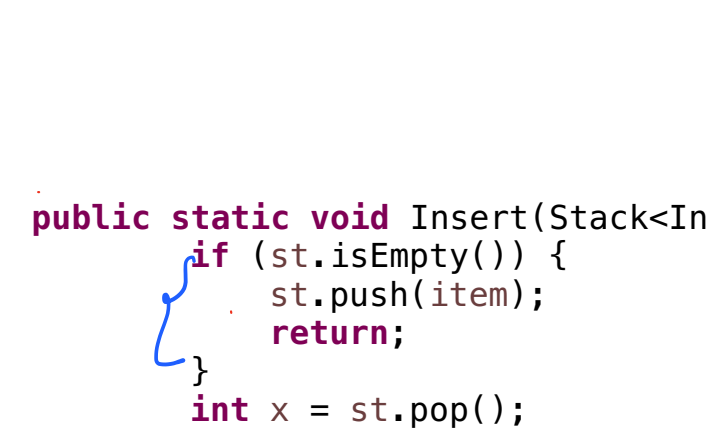
07/07/26 10:21 AM



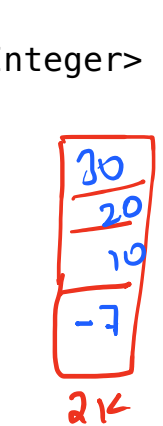
```java
public static void main(String[] args) {
    // TODO Auto-generated method stub
    Stack<Integer> st = new Stack<>();
    st.push(10);
    st.push(20);
    st.push(30);
    st.push(40);
    st.push(50);
    System.out.println(st);
    Insert(st, -7);
    System.out.println(st);
}

public static void Insert(Stack<Integer> st, int item) {

}
```

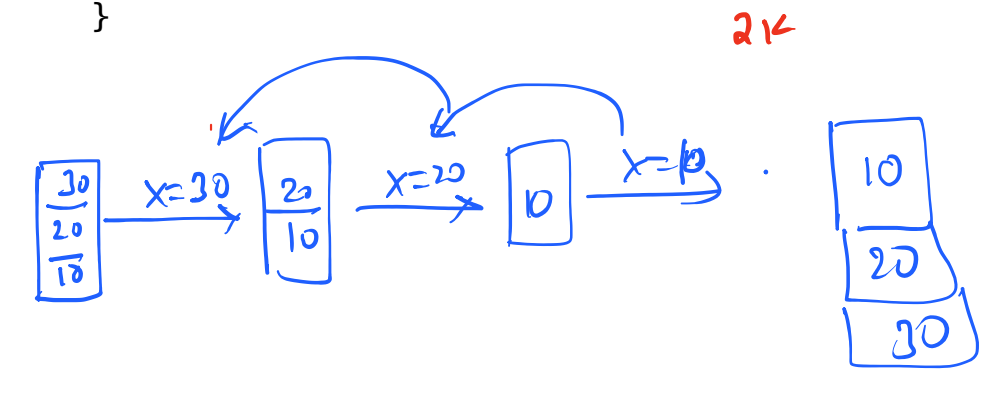

10
40
30
20
10
-7
X
10
20
30
40
50

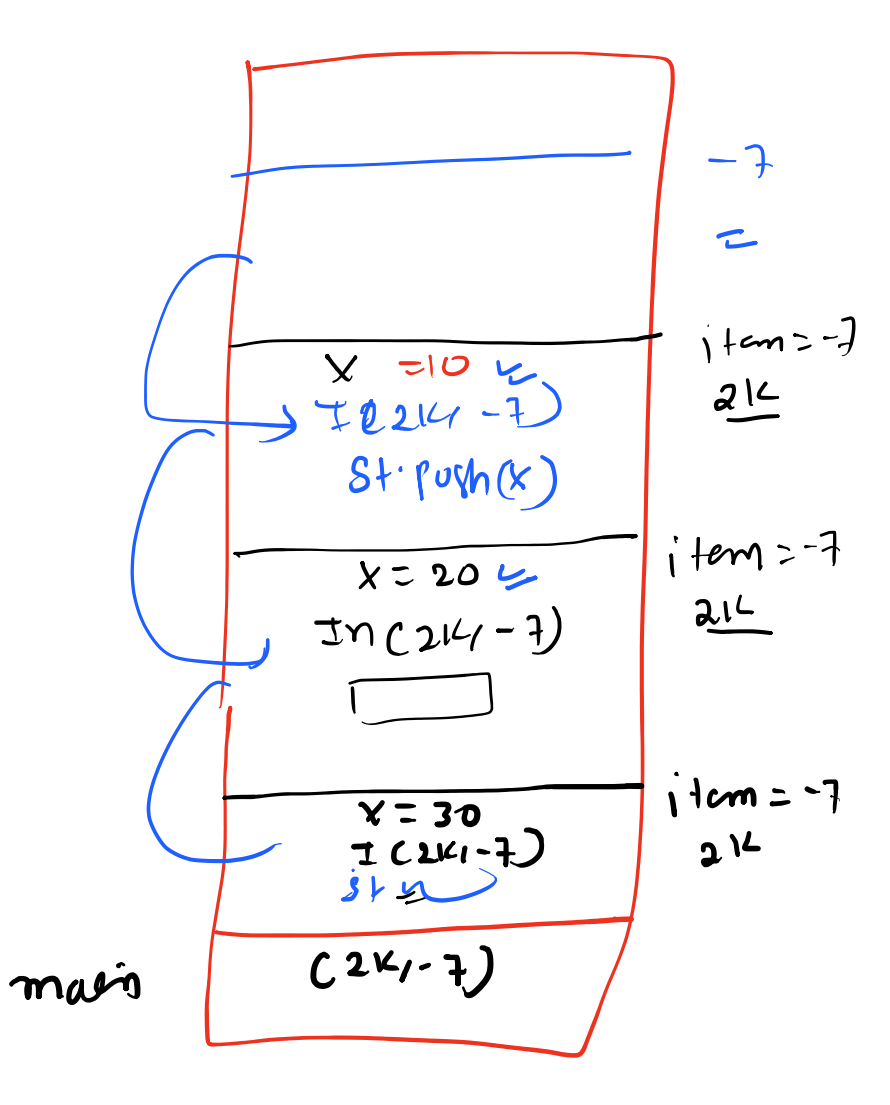

30
20
10
x=30
20
10
x=20
10
x=10
30
20
10
-7

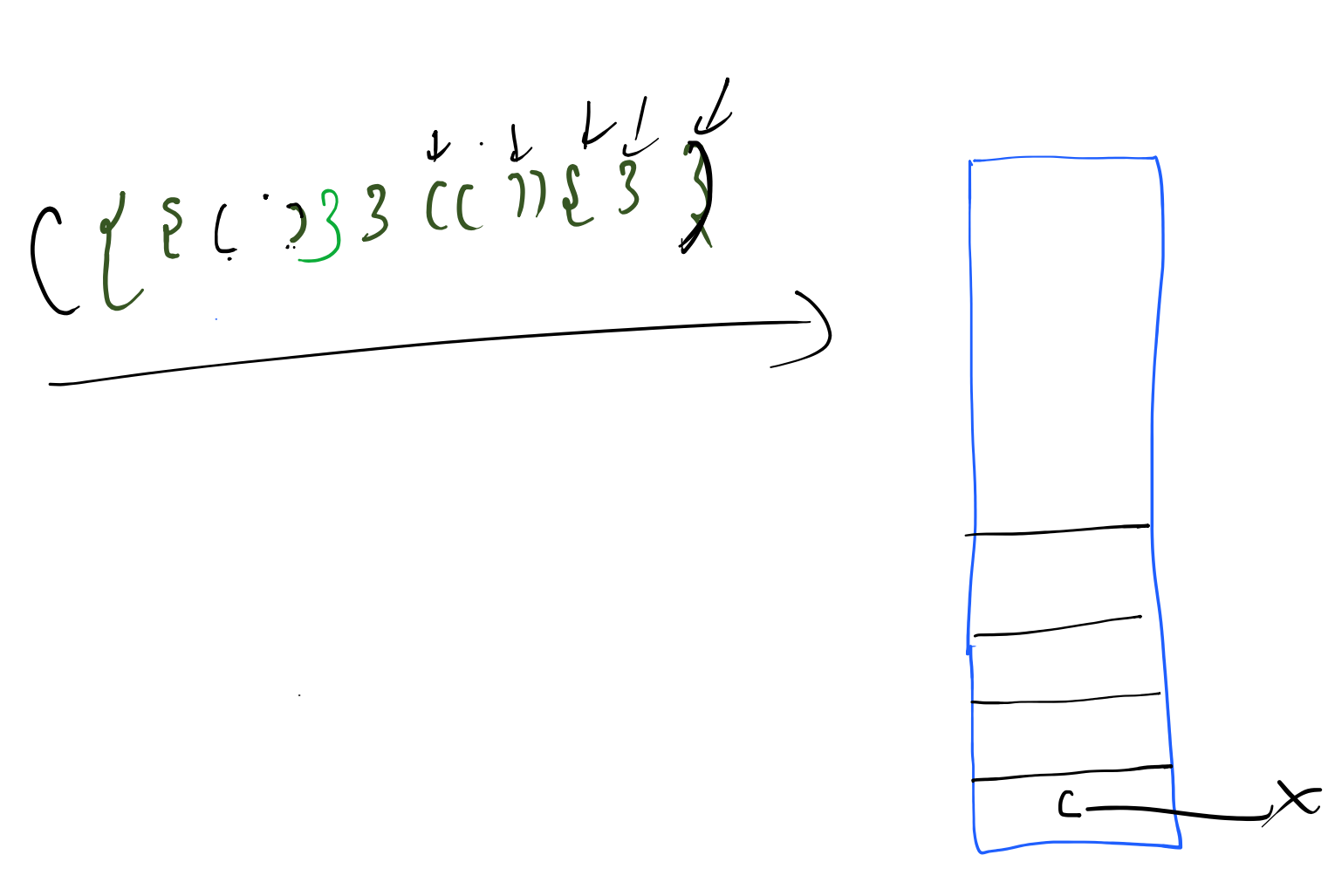


```java
public static void Insert(Stack<Integer> st, int item) {
    if (st.isEmpty()) {
        st.push(item);
        return;
    }
    int x = st.pop();
    Insert(st, item);
    st.push(x);
}
```

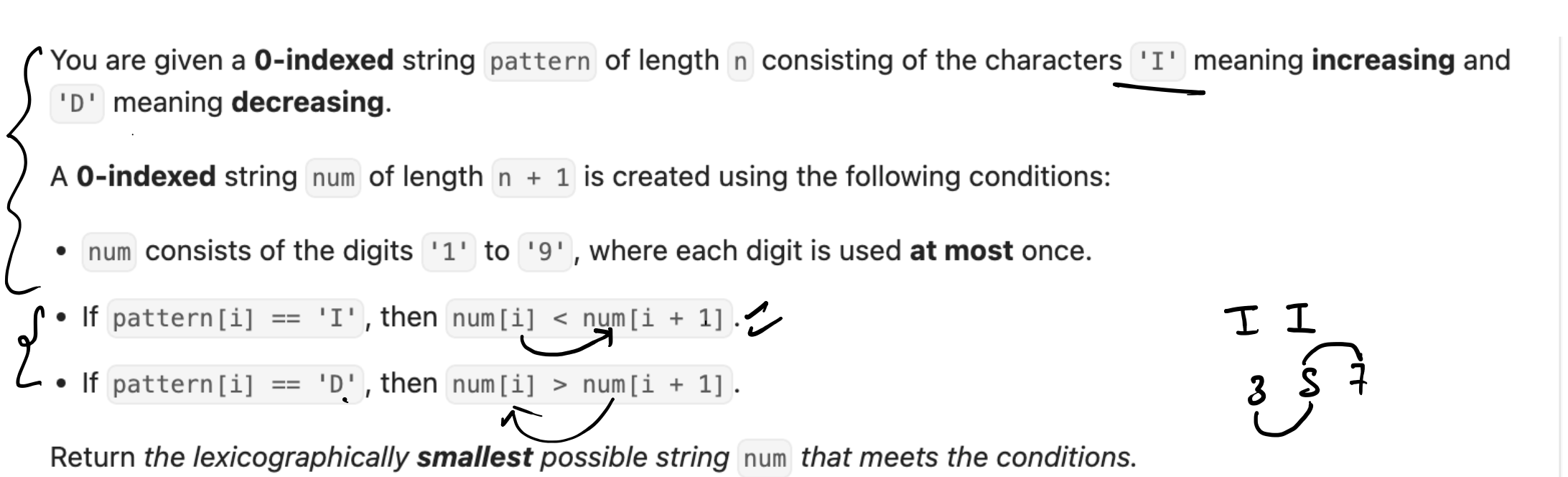



30
20
10
x=30
20
10
x=20
10
x=10
10
20
30


-7
x=10
I(2k,-7)
st.push(x)
item=-7
2k
x=20
In(2k,-7)
item=-7
2k
x=30
I(2k,-7)
st.push
item=-7
2k
main
(2k,-7)



why stack ? Empty (←



You are given a **0-indexed** string `pattern` of length `n` consisting of the characters `'I'` meaning **increasing** and `'D'` meaning **decreasing**.

A **0-indexed** string `num` of length `n + 1` is created using the following conditions:

- `num` consists of the digits `'1'` to `'9'`, where each digit is used **at most** once.
- If `pattern[i] == 'I'`, then `num[i] < num[i + 1]`.
- If `pattern[i] == 'D'`, then `num[i] > num[i + 1]`.

Return the *lexicographically* ***smallest*** *possible string* `num` *that meets the conditions.*

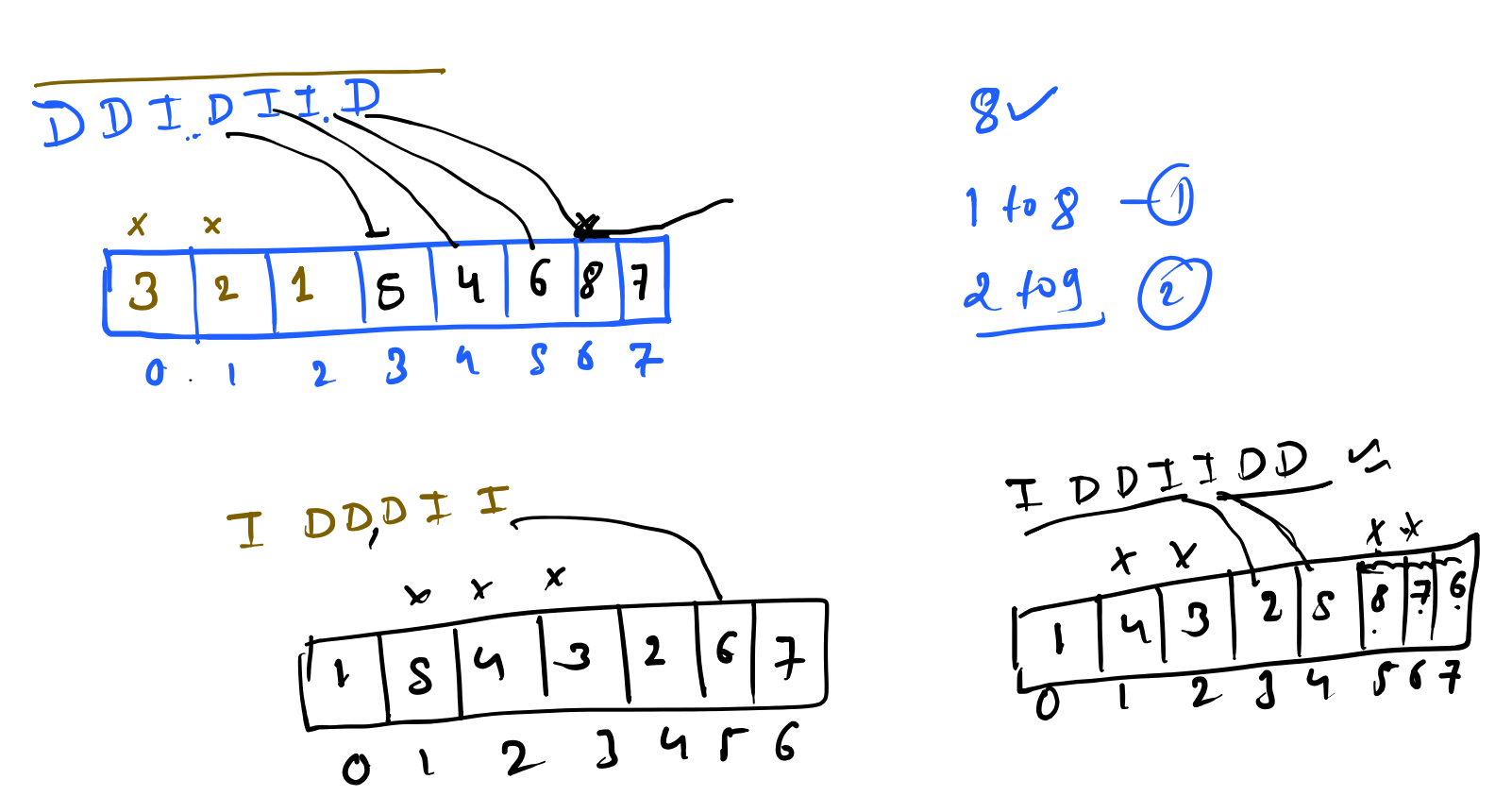
IIII
12345

III
2356
6789
2789
(1234)

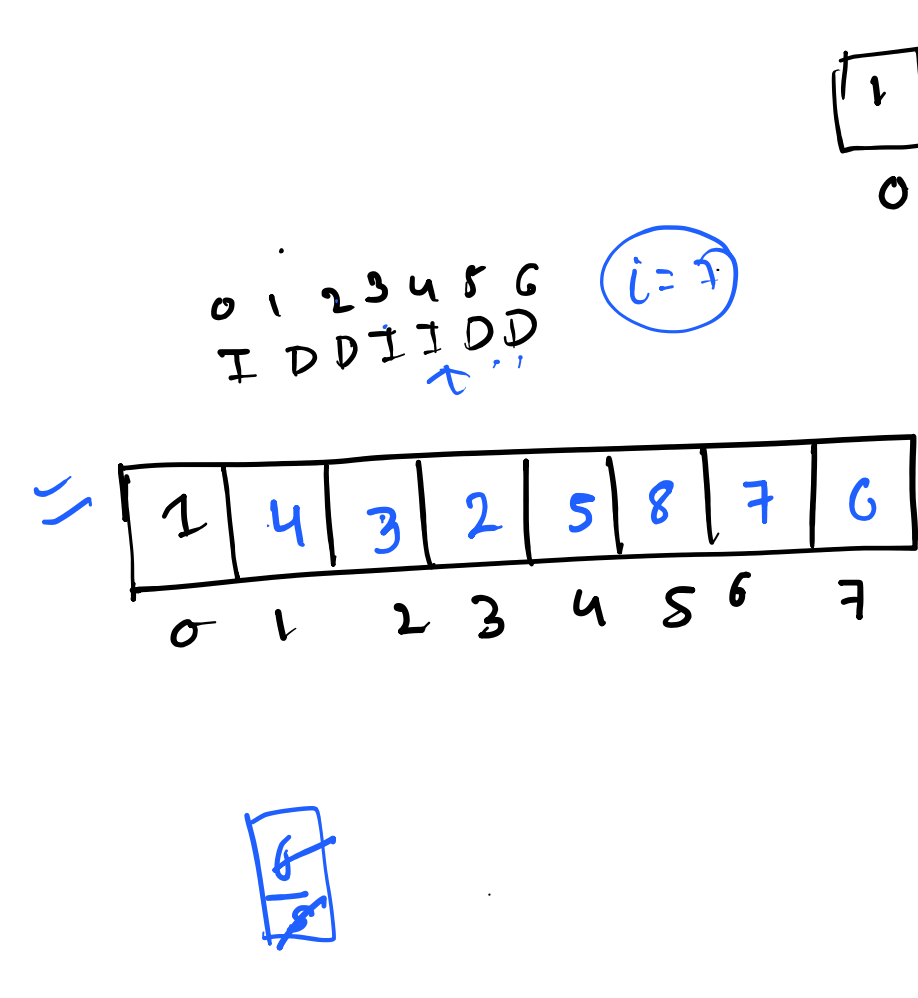
II
357

DD
421

DDD
9876
8432
(4321)


DDIDIIID
3 2 1 5 4 6 8 7
0 1 2 3 4 5 6 7
8
1 to 8 ①
2 to 9 ②
I DDDI I
1 5 4 3 2 6 7
0 1 2 3 4 5 6
I DDI I DD
1 4 3 2 5 8 7 6
0 1 2 3 4 5 6 7

0 1 2 3 4 5 6
I D D I I D D (i=7)


1 4 3 2 5 8 7 6
0 1 2 3 4 5 6 7

int c=1 2 3 4 5 6 7 8 9

for(i=0 i<=s.len() i++){

if(i==s.len() || s.charAt(i)=='I'){

ans[i]=c++

while(!st.isEmpty()){

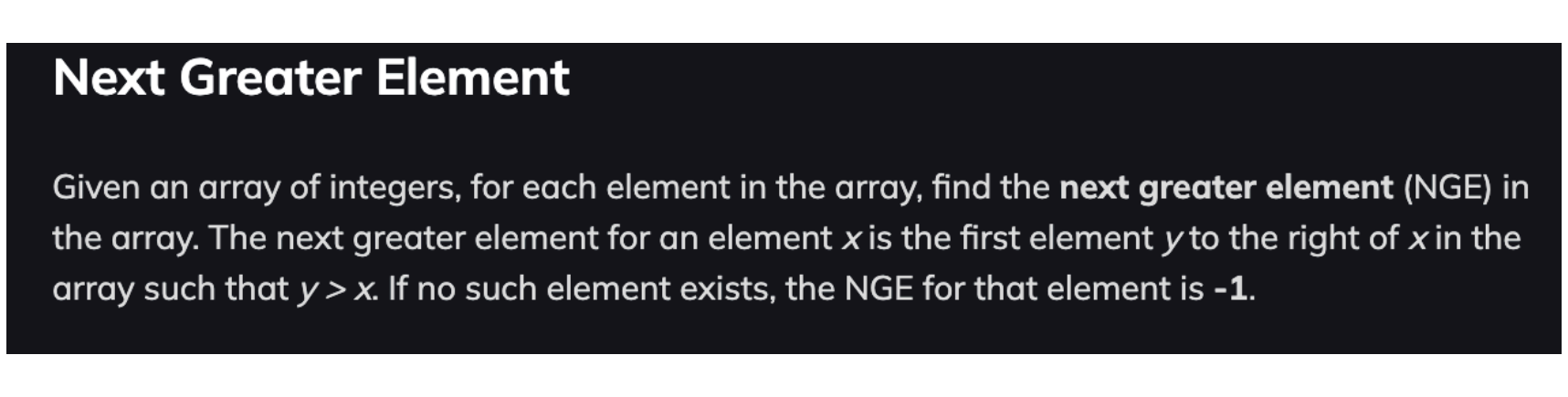
ans(st.pop()) = c++

}

}

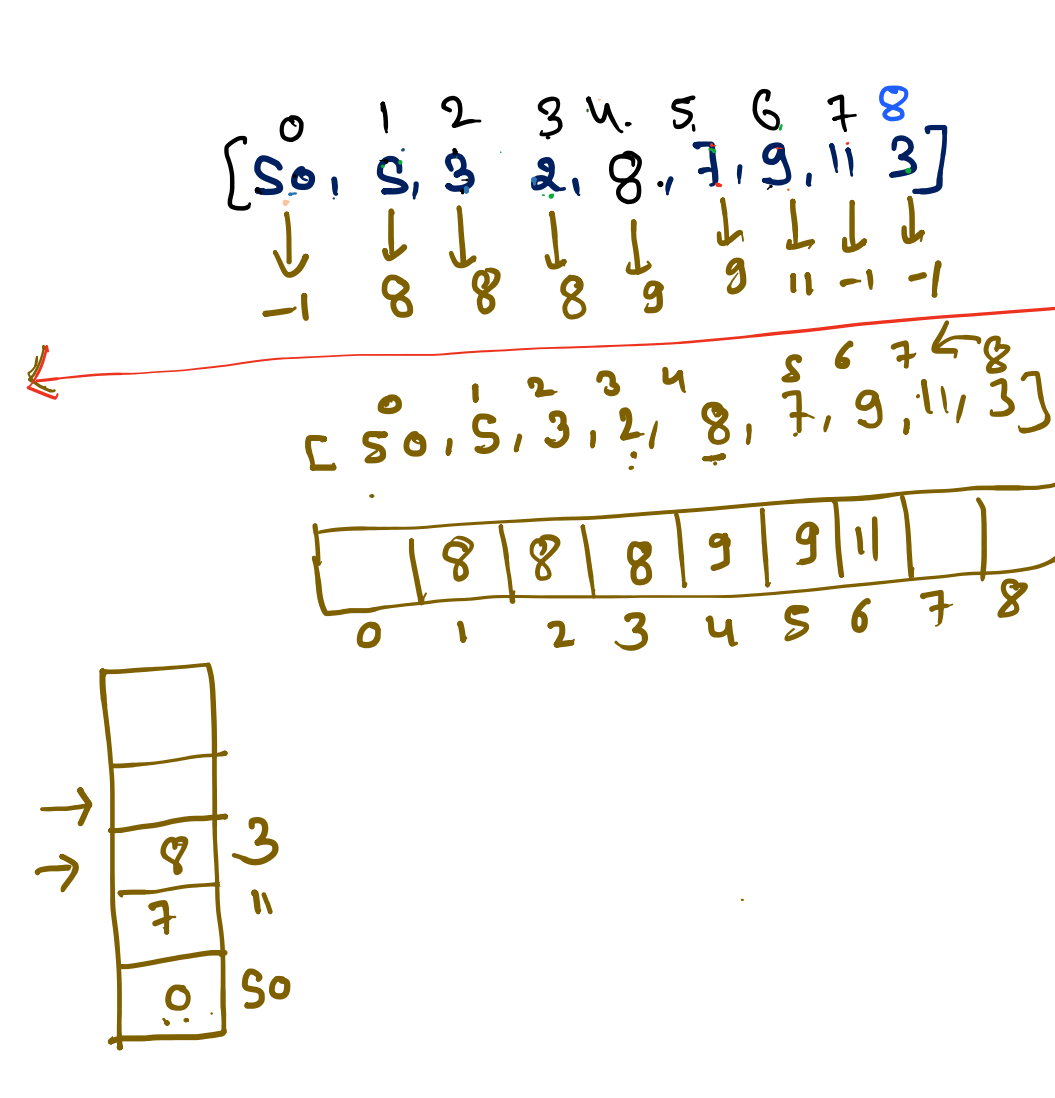
st.push(i)

}



Next Greater Element

Given an array of integers, for each element in the array, find the **next greater element** (NGE) in the array. The next greater element for an element *x* is the first element *y* to the right of *x* in the array such that *y* > *x*. If no such element exists, the NGE for that element is **-1**.


0 1 2 3 4 5 6 7 8
[50, 5, 3, 2, 8, 7, 9, 11, 3]
-1 8 8 8 9 9 11 -1 -1
[50, 5, 3, 2, 8, 7, 9, 11, 3]
8 8 8 9 9 11
0 1 2 3 4 5 6 7 8
8 3
7 11
0 50

for(i=0 i<arr.len i++){

while(!st.isEmpty() && arr[i] > arr[st.peek()]){

ans[st.pop()] = arr[i]

}

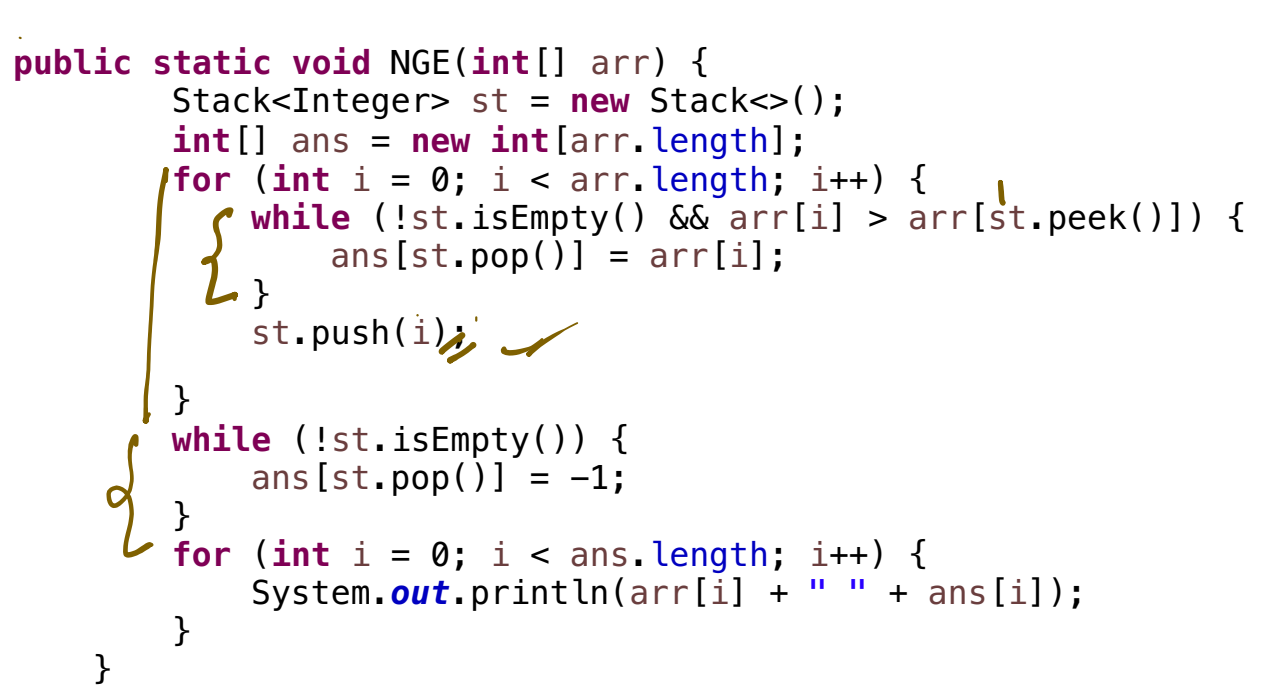
st.push(i)

}

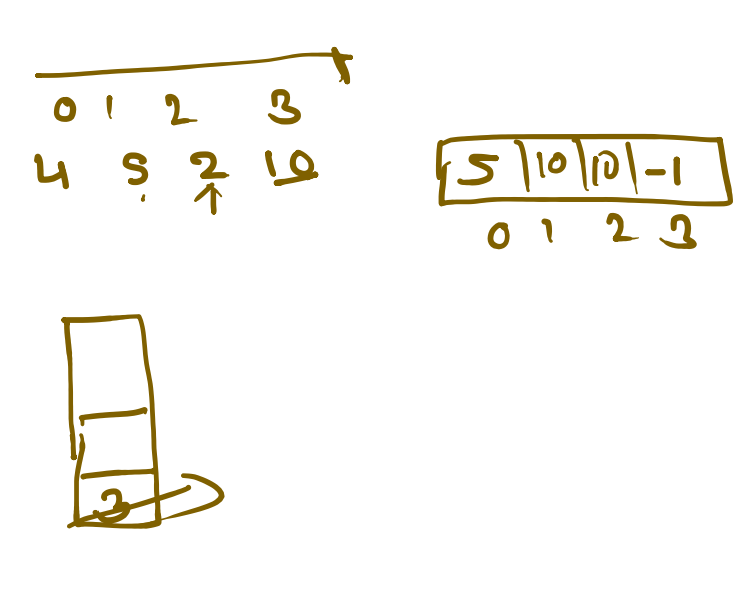
while(!st.isEmpty()){

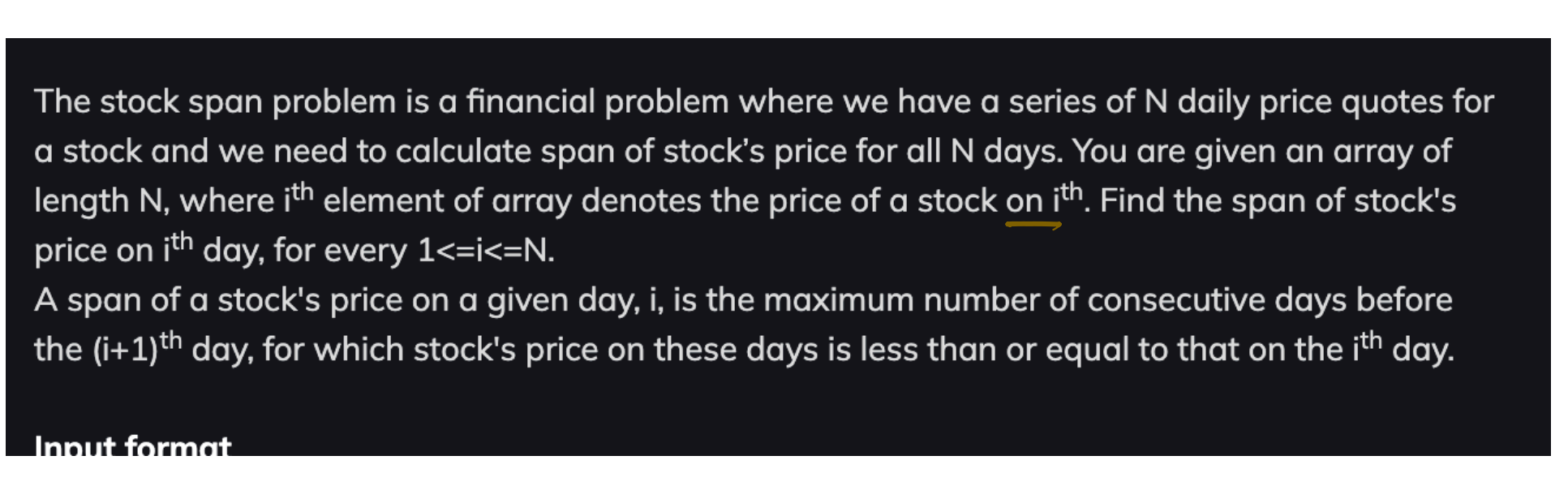
ans(st.pop()) = -1

}



```java
public static void NGE(int[] arr) {
    Stack<Integer> st = new Stack<>();
    int[] ans = new int[arr.length];
    for (int i = 0; i < arr.length; i++) {
        while (!st.isEmpty() && arr[i] > arr[st.peek()]) {
            ans[st.pop()] = arr[i];
        }
        st.push(i);
    }
    while (!st.isEmpty()) {
        ans[st.pop()] = -1;
    }
    for (int i = 0; i < ans.length; i++) {
        System.out.println(arr[i] + " " + ans[i]);
    }
}
```


0 1 2 3
4 5 7 10
5 10 10 -1
0 1 2 3

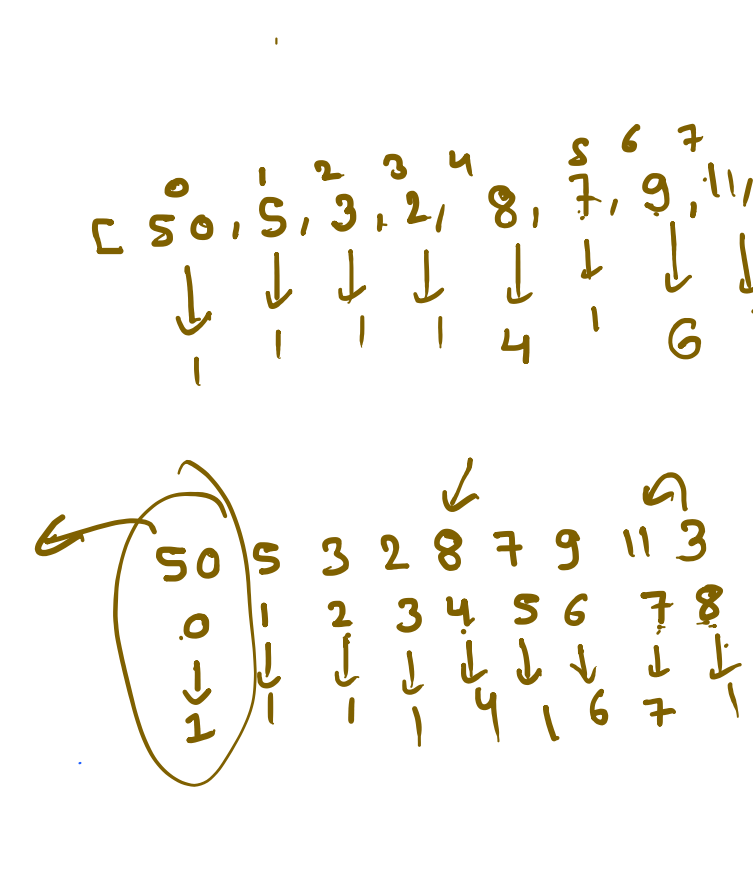


The stock span problem is a financial problem where we have a series of N daily price quotes for a stock and we need to calculate span of stock's price for all N days. You are given an array of length N, where ith element of array denotes the price of a stock on ith. Find the span of stock's price on ith day, for every 1<=i<=N.

A span of a stock's price on a given day, i, is the maximum number of consecutive days before the (i+1)th day, for which stock's price on these days is less than or equal to that on the ith day.

Input format

0 1 2 3 4
30 34 40 38 35
1 2 3 1 1


[50, 5, 3, 2, 8, 7, 9, 11, 3]
1 1 1 1 4 1 6 7 1
50 5 3 2 8 7 9 11 3
0 1 2 3 4 5 6 7 8
8 3
0 50

for(i=0 i<n i++){

while(!st.isEmpty() && arr[i] >= arr[st.peek()]){

st.pop()

}

if(st.isEmpty()){

ans[i]=i+1

}

else{

ans[i] = i-st.peek()

}

st.push(i)

}

0 1 2 3 4 5 6
[2, 3, 5, 4, 6, 1, 7]

2×5 =10

3×4 =12

5×1 = 5

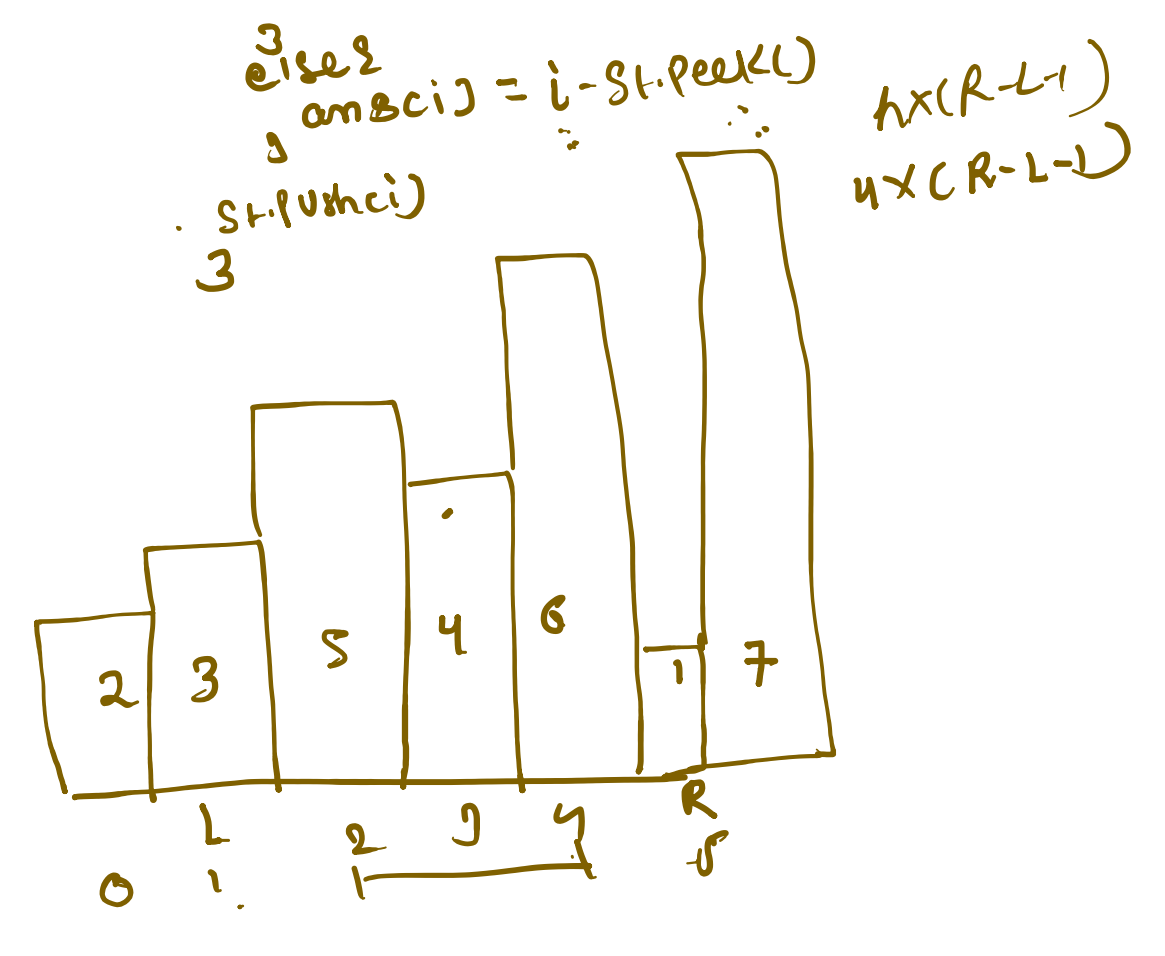
4×3 = 12

6×1 = 6

1×7 = 7

7×1 = 7

(12)


h×(R-L-1)
4×(R-L-1)
2 3 5 4 6 1 7
0 1 2 3 4 R 5

0 1 2 3 4 5 6
[2, 3, 5, 4, 6, 1, 7]

7 → 0

R = arr.length

(h×R)

(12)

h×(R-L-1)

5×(3-1-1) = 5

6×(5-3-1) = 6

4×(5-1-1) = 12

3×(5-0-1) = 12

2×R → 2×5 = 10

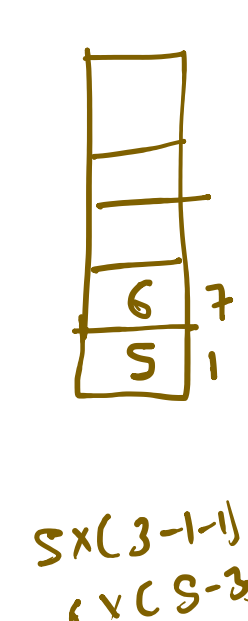
7×(7-5-1) = 7

1×7 = 7

7×(7-5-1) = 7

h=5, R=3

0 1 2 3 4 5 6
[2, 3, 5, 4, 6, 1, 7]



R = arr.len

while(!St.isEmpty()){

h = arr[St.pop()]

if(St.isEmpty()){

area = h×R

}

else{

l = St.peek

area = max(area, h×(R-l-1))

}

for(i=0 i<arr.len i++){

while(!st.isEmpty() && arr[i] < arr[st.peek()]){

R=i, h=arr[st.pop()]

if(st.isEmpty()){

area = max(area, h×R)

}

else{

l = st.peek

area = max(area, h×(R-L-1))

7×(7-5-1)

(7)

}

St.push(i)

5×(3-1-1) = 5

6×(5-3) = 6

4×(5-1-1) = 12

3×(5-0-1) = 12

2×5 = 10

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

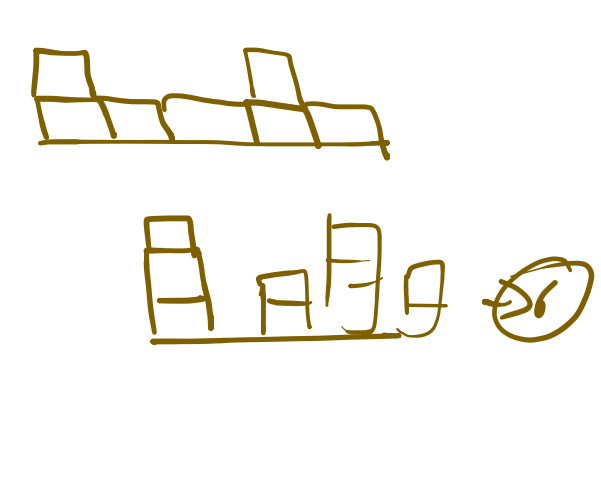
→ [1,0,1,0,0]

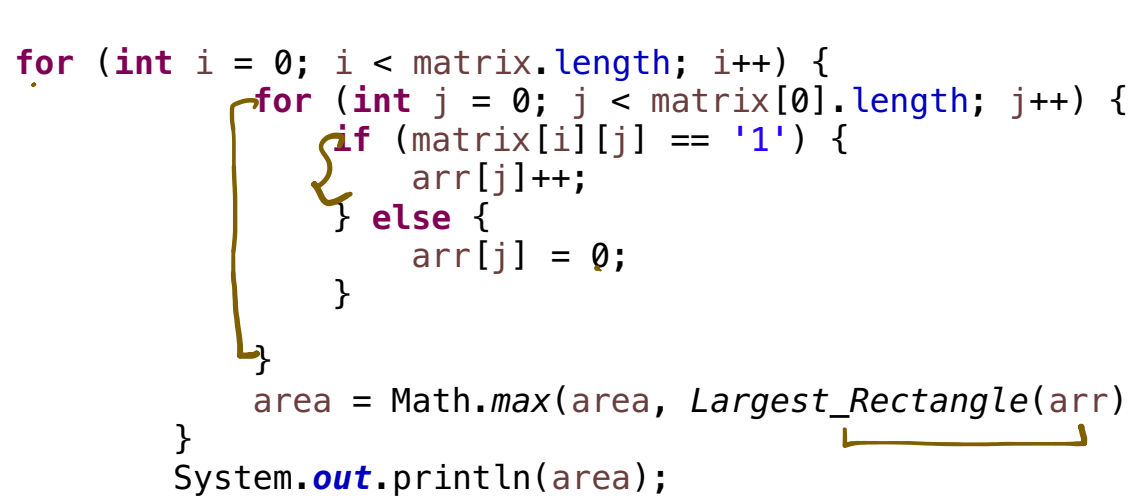
[1, 0, 0, 1, 0]

[2, 1, 1, 2, 1] → 5

[3, 0, 2, 3, 2]

(4, 0, 3, 0, 0)





```java
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[0].length; j++) {
        if (matrix[i][j] == '1') {
            arr[j]++;
        } else {
            arr[j] = 0;
        }
    }
    area = Math.max(area, Largest_Rectangle(arr));
}
System.out.println(area);
```

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

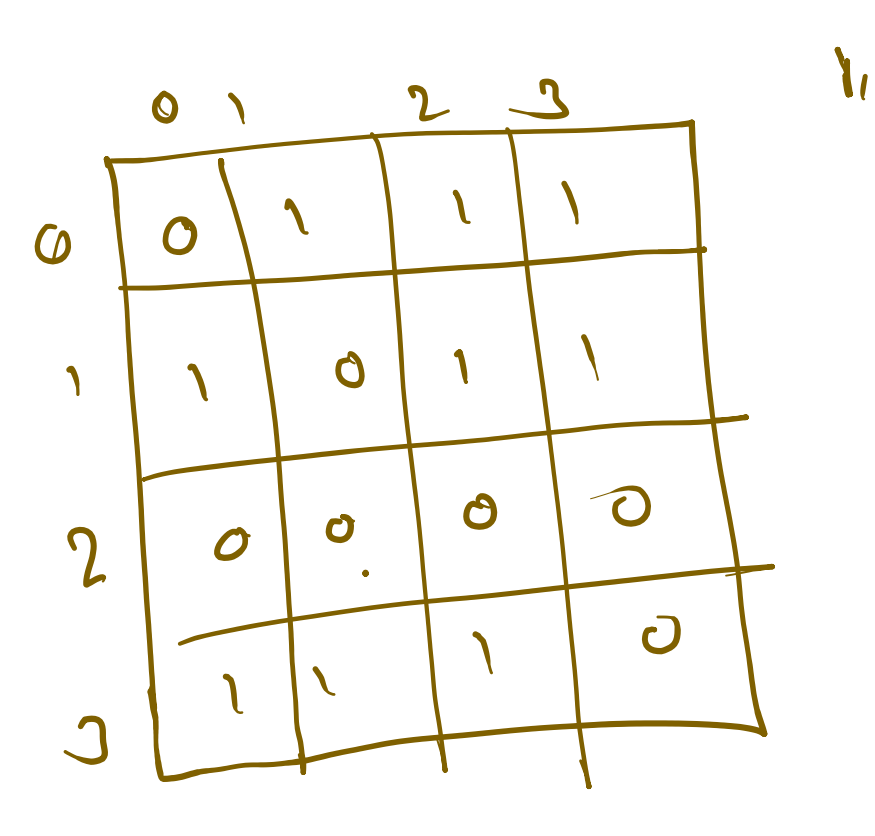
[1,0,1,0,0]

area = 1 3 6

[2, 0, 2, 1, 1]

[3, 1, 3, 2, 2]

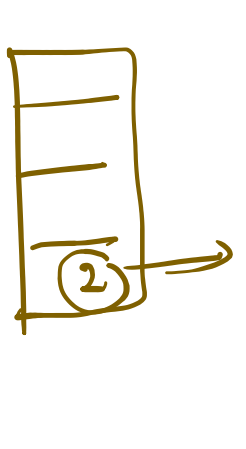
(4, 0, 0, 3, 0)


0 1 2 3
0 0 1 1 1
1 1 0 1 1
2 0 0 0 0
3 1 1 1 0

1, can → 0 || cm (→1

0-2, 1-2, 3-2 → 0

2-0, 2-1, 2-3 → 1



a = 3 b = 2

a = 2, b = 1

a = 2 b = 0

A container has milk and water in the ratio 5:2. If 14 litres of mixture is removed and replaced with water, the new ratio becomes 8:5. Find the original quantity of mixture.

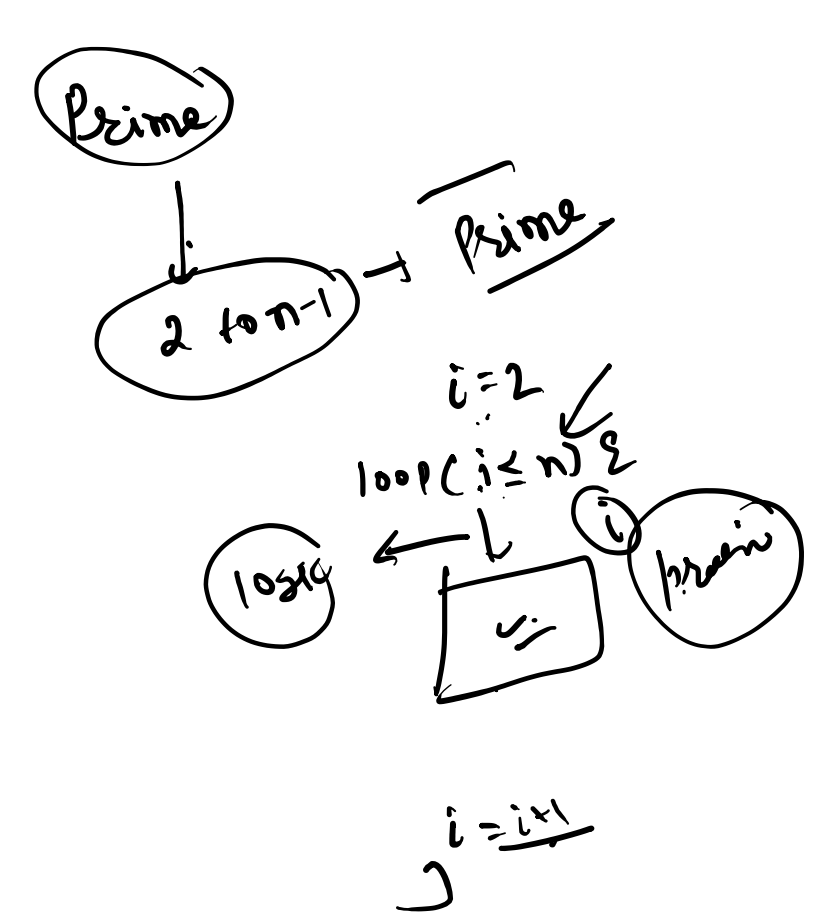
Three dice are thrown. What is the probability that exactly two dice show the same number?

Three dice throw :

Dice 1 Dice 2 Dice 3
Ab 4 exactly **2 dice** par Hai.
Possibilities:

Dice 1	Dice 2	Dice 3
4	4	_
4	_	4
_	4	4

Repeated number choose karo = 6 ways
Repeated number ki positions choose karo = 3 ways
Third dice par different number choose karo = 5 ways


Prime
2 to n-1
Prime
i=2
loop(i<n){
loose
prime
i=i+1